

Majed Hmoud

Event Booking · Weekly Mini Project 02 · Team 4

Backend

Responsibility: Events Module, Seed Data and Availability Calculations

You own everything about events: the data, the seeded demo content, the single function that computes availability, and the endpoints that read events. Adham's booking logic depends on your calculation function and your in-memory store, so your first phases unblock him.

You provide

- The in-memory store and shared types (to Adham, Allaith)
- The availability calculation function (to Adham)
- Working GET /events endpoints (to Taima, Badr)
- A list of seeded demo bookings, phone + code pairs (to Yousef, for the demo)

You depend on

- Adham: agree together on the shared store and the Booking type in Phase 1
- Allaith: the final API contract

Working rule: all business rules are enforced in the backend. If something is unclear or the contract needs a change, tell the team leader (Allaith) so every file stays consistent.

Your Flow, Phase by Phase

Phase 1: Project Foundation (shared with Adham)

Make one working NestJS app that both of you can build on without conflicts.

- ☐ Create the NestJS project, run it, and push it so Adham can pull it. Agree on one folder layout (for example: events/, bookings/, store/, common/).
- ☐ Create the shared TypeScript types: Event, Booking, EventStatus, BookingStatus, exactly matching the contract fields.
- ☐ Create a StoreService (injectable, exported) that holds two in-memory arrays: events and bookings. Both modules use this one service; nobody keeps a second copy of the data.
- ☐ Do not store booked or remaining on the event. They are always computed from bookings, so they can never disagree.
- ☐ Create one time helper, for example hasStarted(event, now), that combines date + time into a start moment. An event whose start is now or earlier is Past. Everyone uses this helper; nobody compares dates on their own.

Phase 2: Availability Calculation Function

Write the one function that every screen's numbers come from.

- ☐ Write computeAvailability(event, bookings, now) returning booked, remaining, bookingPercentage and status.
- ☐ booked = sum of places of bookings with status Active for that event only. Cancelled bookings never count.
- ☐ remaining = capacity minus booked. bookingPercentage = booked / capacity * 100, rounded to a whole number.
- ☐ Status order of checks: Past first (event has started), then Full (remaining is 0), then Almost full (percentage is 90 or more), otherwise Available.
- ☐ Write a small helper that returns an Event ready for the response (the stored event fields plus the computed ones). Adham will reuse it for the booking responses.
- ☐ Send the function to Adham as soon as it works, even before the endpoints exist.

Phase 3: Seed Data

Prepare demo data that lets every business rule be demonstrated.

- ☐ At least 10 events across at least 4 categories (for example Workshop, Talk, Concert, Sports, Meetup), with different capacities (small like 10 and larger like 100).
- ☐ Generate dates relative to today (for example today + 3 days) instead of fixed dates, so upcoming events stay upcoming and the Past event stays past whenever the demo is run.
- ☐ Include at least: one **Full** event (active seeded bookings add up exactly to capacity), one **Almost full** event (90% or more, but not full), one **Past** event, and several Available events with different fill levels.

- ☐ Include a Workshop with 3 places remaining, similar to the specification's example ("UI Design Basics", capacity 30, 27 booked).
- ☐ Create seeded bookings with valid phone numbers, unique codes in the EV-XXXX format, realistic names, and at least two seeded Cancelled bookings to prove they are not counted.
- ☐ Make sure no seeded phone has two Active bookings for the same event.
- ☐ Because booked is computed, the numbers match automatically. Still, print or check every event once to confirm booked + remaining = capacity.
- ☐ Write down 3 demo phone + code pairs (one Active that can be cancelled, one on the Past event, one Cancelled) and give them to Yousef for the demo.

Phase 4: Events Endpoints

Expose the events to the frontend.

- ☐ GET /events: by default return only upcoming events, sorted by date then time ascending. Each item includes the computed fields.
- ☐ Filters (all optional, all combinable, applied together): category (case-insensitive exact match), date (YYYY-MM-DD, events on that day), availableOnly=true (excludes Full and Past), includePast=true (also returns past events, marked with status Past).
- ☐ Validate the query: a wrong date format returns 400 with a clear message such as "Date must be in YYYY-MM-DD format".
- ☐ GET /events/categories: the list of distinct category names, used by the frontend filter dropdown.
- ☐ GET /events/:id: one event with all computed fields; if it does not exist return 404 with the message "Event not found".
- ☐ Use the shared exception shape from Adham's filter so errors look identical everywhere.

Phase 5: Testing and Handover

Prove the numbers are right and make the frontend's life easy.

- ☐ Test every filter alone and in combination (category + date, category + availableOnly, all three) and the empty result case.
- ☐ Test that Past events never appear by default and do appear with includePast=true.
- ☐ After Adham's booking endpoints exist, verify the full loop: book, then check that remaining dropped on /events and /events/:id; cancel, then check it went back up.
- ☐ Prepare a Postman collection (or a small list of curl commands) for all events requests and share it with Taima and Badr.
- ☐ Tell the team the seeded phone + code pairs and the IDs of the Full, Almost full and Past events.

Phase 6: Presentation Preparation

Prepare your speaking part in English.

- ☐ Topic: the business rules about capacity, time and event status, and how a user experiences them (event fills up, Full status, Past events are not bookable).
- ☐ Prepare 3 to 4 simple English sentences per rule, in your own words, and practice them out loud.
- ☐ Be ready to explain in one minute why booked and remaining are computed instead of stored.

Definition of Done

- ☐ 10+ events, 4+ categories, Full / Almost full / Past all present, seeded bookings consistent
- ☐ booked + remaining = capacity for every event, always
- ☐ Filters work alone and combined; past events hidden by default
- ☐ All error messages are readable sentences
- ☐ Postman collection shared; demo phone + code pairs handed to Yousef

Your Part in the Final English Presentation

Business rules: capacity, time, and event status (Flow C: an event fills up).

Appendix: API Contract Reference

Route	Input	Success	Owner
GET /events	query: category, date (YYYY-MM-DD), availableOnly=true, includePast=true	200 + Event[]	Majed
GET /events/categories	none	200 + string[]	Majed
GET /events/:id	none	200 + Event	Majed
POST /bookings	body: eventId, customerName, customerPhone, places	201 + Booking	Adham
POST /bookings/find	body: phone, code	200 + Booking	Adham
POST /bookings/:id/cancel	body: phone	200 + Booking	Adham

Event: id, title, description, date (YYYY-MM-DD), time (HH:mm), location, category, capacity, booked, remaining, bookingPercentage, status

status: "Available" | "Almost full" | "Full" | "Past"

Booking: id, code (EV-XXXX), eventId, event (full Event, with updated availability), customerName, customerPhone, places, bookedAt, status ("Active" | "Cancelled"), canCancel (boolean)

Error shape (every failure): { statusCode, message }, where message is one readable sentence ready to show to the user. Codes: 400 invalid input, 404 not found, 409 rule conflict (full, not enough places, duplicate, past event, already cancelled, event started).

This contract is a proposal. The team leader confirms the final version; if a field name changes, the leader updates this reference.